

Práctica 2: TEORÍA DE ERRORES

Objetivos

1. Trabajar computacionalmente las nociones vistas en Enseñanzas Básicas correspondientes a la Teoría de Errores, utilizando MATHEMATICA.
2. Conocer los comandos MATHEMATICA que permiten realizar el tratamiento computacional anteriormente indicado.

Contenidos

1. Cancelaciones

Dependiendo de la precisión/exactitud con la que estemos trabajando nos puede interesar realizar cancelaciones automáticas en operandos que aparezcan en la expresión a estudiar. Para ello, bastaría con utilizar el comando `Chop` como sigue:

`Chop[expr]`: reemplaza por 0 todo valor en `expr` de magnitud menor que 10^{-10} .

```
In[1]:=Chop[25.0001`7+1.22222`7×10-12x2]  
Out[1]=25.00010
```

`Chop[expr, dx]`: reemplaza por 0 todo valor en `expr` de magnitud menor que `dx`.

```
In[2]:=Chop[15.7777`9+0.0257`9-0.00003`9,10-3]  
Out[2]=15.8033700  
In[3]:=15.7777`9 + 0.0257`9  
Out[3]=15.8034000
```

2. Representación numérica respecto de una base

`IntegerDigits[n]`: lista de los dígitos decimales para el número entero `n`.

```
In[4]:=IntegerDigits[58127]  
Out[4]={5,8,1,2,7}  
In[5]:=5*104+8*103+1*102+2*101+7*100  
Out[5]=58127
```

`IntegerDigits[n,b]`: lista de los dígitos para el número entero `n` en base `b`.

```
In[6]:=IntegerDigits[58127, 2]
Out[6]={1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1}
In[7]:=2^15+2^14+2^13+2^9+2^8+2^3+2^2+2^1+2^0
Out[7]=58127
```

De hecho, el comando `IntegerDigits` puede usarse sobre varios números a la vez y respecto de varias bases simultáneamente. Esto es lo que vemos en los dos ejemplos siguientes: en primer lugar, obtenemos la representación en binario (base 2) de los números 6, 7 y 2; el segundo ejemplo corresponde a escribir el número 7 con respecto de las bases 2, 3 y 4:

```
In[8]:=IntegerDigits[{6, 7, 2},2]
Out[8]={{1, 1, 0}, {1, 1, 1}, {1, 0}}
In[9]:=IntegerDigits[7,{2, 3, 4}]
Out[9]={{1,1,1},{2,1},{1,3}}
```

`FromDigits[lista]`: devuelve el número que tiene por dígitos decimales a los que se indican en la `lista` siendo el primero el de la mayor potencia de 10 y el último el de la potencia $10^0 = 1$. Es decir, el comando `FromDigits` viene a ser el inverso del comando `IntegerDigits`.

```
In[10]:=FromDigits[{5,8,1,2,7}]
Out[10]=58127
```

`FromDigits[lista,b]`: devuelve el número que tiene por dígitos respecto de la base `b` a los que se indican en la `lista` siendo el primero el de la mayor potencia de `b` y el último el de la potencia $b^0 = 1$.

```
In[11]:=FromDigits[{1, 0, 1, 1, 0, 1, 1},2]
Out[11]=91
In[12]:=2^6 + 2^4 + 2^3 + 2 + 1
Out[12]=91
```

Finalizamos esta sección, mostrando la escritura formal que tendría un desarrollo en base `b`:

```
In[13]:=Expand[FromDigits[{a4, a3, a2, a1, a0},b]]
Out[13]=a0 + a1 b + a2 b^2 + a3 b^3 + a4 b^4
```

3. Representación computacional de números

`NumberForm[expr,n]`: este comando devuelve una aproximación de `expr` (que debe ser decimal) con precisión de `n` dígitos. Veamos qué pasaría con una aproximación del número 1.7176627 y del número e que tenga la precisión de 4 dígitos:

```

In[14]:=NumberForm[1.7176627,4]
Out[14]//NumberForm=
1.718
In[15]:=NumberForm[N[E,10],4]
Out[15]//NumberForm=
2.718

```

NumberForm[expr,{n,m}]: esta variante devuelve una aproximación de **expr** (que debe ser decimal) con precisión de **n** dígitos y con **m** cifras a izquierda del punto decimal. En caso de no disponerse de suficientes cifras significativas, Mathematica añade tantos ceros como fuesen necesarios a la derecha del último dígito dado por la precisión. Cojamos los dos ejemplos anteriores con precisión de 3 dígitos, pero con 4 cifras a la derecha del punto decimal:

```

In[16]:=NumberForm[1.7176627,{3,4}]
Out[16]//NumberForm=
1.7200
In[17]:=NumberForm[N[E,10],{3,4}]
Out[17]//NumberForm=
2.7200

```

ScientificForm[expr]: devuelve el número **expr** (que debe ser decimal) en notación científica.

```

In[18]:=ScientificForm[{123450000.0,0.000012345,123.45}]
Out[18]//ScientificForm=
{1.2345×108,1.2345×10-5,1.2345×102}

```

ScientificForm[expr,n]: devuelve el número **expr** (que debe ser decimal) en notación científica usando una precisión de **n** dígitos. Veamos las aproximaciones anteriores utilizando precisión de 3 dígitos.

```

In[19]:=ScientificForm[{123450000.0,0.000012345,123.45},3]
Out[19]//ScientificForm=
{1.23×108,1.23×10-5,1.23×102}

```

BaseForm[expr,n]: devuelve el número **expr** (que debe ser decimal) expresado en base **b**. Veamos cómo se traduciría un número de su forma decimal a su expresión en base 2:

```

In[20]:=BaseForm[33,2]
Out[20]//BaseForm=
1000012
In[21]:=BaseForm[0.1,2]
Out[21]//BaseForm=
0.000110011001100110011012

```

4. Exponente y mantisa de un número

El comando `MantissaExponent[num]` da una lista con dos argumentos, el primero correspondiente a la mantisa de `num` y el segundo al exponente. Veámoslo con dos ejemplos:

```
In[22]:=MantissaExponent[3.4×10^30]
Out[22]={0.34, 31}
In[23]:=MantissaExponent[456.1414]
Out[23]={0.456141, 3}
```

Si añadimos un segundo argumento `b` al comando `MantissaExponent`, lo que estamos es indicando la base con respecto del cuál queremos calcular la mantisa y el exponente.

```
In[24]:=MantissaExponent[1027, 2]
Out[24]={ $\frac{1027}{2048}$ , 11}
In[25]:= {m,e}=MantissaExponent[N[Pi,20],E]
Out[25]={0.42516833158763632844, 2}
In[26]:=Precision[m]
Out[26]=20.
```

Observa que la precisión de la mantisa es la del número al que se le calcula la mantisa.

5. Desbordamientos

Cuando trabajamos computacionalmente, no todos los números son manejables por el ordenador. Cuando sobrepasamos la capacidad para representar números, entonces estamos ante los desbordamientos. Son dos valores numéricos los que causan un desbordamiento tanto por exceso como por defecto.

Mathematica trabaja con dos límites numéricos para cada uno de los dos tipos de desbordamientos: uno es el límite correspondiente a los números en punto flotante (precisión de la máquina); y el otro es el límite de los números de precisión arbitraria que admite Mathematica. Indicar que la ventaja que presenta Mathematica es que el desbordamiento no se tiene por superar los límites para la precisión de la máquina, sino por superar los de precisión arbitraria, que permiten representar un rango más amplio de valores.

Los valores de estos límites de representación se calculan con Mathematica como sigue:

\$MaxNumber y **\$MinNumber**: corresponden al mayor y menor número de precisión arbitraria que puede representarse con Mathematica en el equipo que se trabaja. Obviamente depende del equipo y del sistema operativo que se emplea.

```
In[27]:=$MaxNumber
Out[27]=5.297557459040040×10323228467
In[28]:=$MinNumber
Out[28]=1.887662394852453×10-323228468
```

Veamos que se llega a un desbordamiento cuando se operan con estos dos valores. Comentar que si operamos con el valor `$MaxNumber` o bien se sigue obteniendo el mismo valor (por ejemplo, sumando números pequeños) o bien se obtiene un overflow (desbordamiento por exceso).

```
In[29]:=$MaxNumber+3
Out[29]=5.297557459040040×10323228467
In[30]:=2$MaxNumber
General::ovfl: Overflow occurred in computation.
Out[30]=Overflow[]
```

Por el contrario, si operamos con el valor `$MinNumber` siempre se obtiene un underflow (desbordamiento por defecto) por ejemplo mediante divisiones o un overflow cuando se suma o se resta.

```
In[31]:=$MinNumber/2
General::unfl: Underflow occurred in computation.
Out[31]=Underflow[]
In[32]:=$MinNumber - 2
General::ovfl: Overflow occurred in computation.
Out[32]=Overflow[]
```

La relación existente entre `$MinNumber` y `$MaxNumber` es que uno es inverso del otro. En efecto:

```
In[33]:=$MaxNumber*$MinNumber
Out[33]=1.000000000000000
```

Además, ninguno de estos dos números tiene precisión de la máquina, sino que su precisión es arbitraria, similar a la de la máquina:

```
In[34]:=MachineNumberQ[$MaxNumber]
Out[34]=False
In[35]:=Precision[$MaxNumber]
Out[35]=15.9546
In[36]:=MachineNumberQ[$MinNumber]
Out[36]=False
In[37]:=Precision[$MinNumber]
Out[37]=15.9546
In[38]:=$MachinePrecision
Out[38]=15.9546
```

`$MaxMachineNumber` y `$MinMachineNumber`: corresponden al mayor y menor número con precisión de la máquina que puede ser usado en el equipo que se trabaja.

```
In[39]:=$MaxMachineNumber
Out[39]=1.79769×10308
In[40]:=$MinMachineNumber
Out[40]=2.22507×10-308
```

Los números que superan tanto el mayor número máquina por exceso como el menor número máquina por defecto pueden representarse por valores de precisión arbitraria, que como hemos visto se mueven en un intervalo de mayor amplitud. Por tanto, después de operar con el valor numérico (mínimo o máximo) con precisión de la máquina, el resultado puede estar fuera del intervalo representable con dicha precisión (por ejemplo, dividiendo el mínimo valor y multiplicando el máximo valor por un número entero). Dichos resultados corresponden a números de precisión arbitraria similar a la de la máquina. Veámoslo con algunos ejemplos:

```
In[41]:=x=2*$MaxMachineNumber
Out[41]=3.595386269724631×10308
In[42]:={MachineNumberQ[x], Precision[x]}
Out[42]={False, 15.9546}
In[43]:=y=$MinMachineNumber/2
Out[43]=1.112536929253601×10-308
In[44]:={MachineNumberQ[y], Precision[y]}
Out[44]={False, 15.9546}
```

\$MachineEpsilon: valor que corresponde a la diferencia entre 1.0 y el número con precisión de la máquina más cercano a él por la derecha (i.e. mayor que 1.0).

```
In[45]:=$MachineEpsilon
Out[45]=2.22045×10-16
In[46]:=Precision[$MachineEpsilon]
Out[46]=MachinePrecision
```

El número **\$MachineEpsilon** es un número por debajo de la precisión de la máquina. Por tanto, cualquier cálculo que se realice con precisión de la máquina despreciará este valor y no lo considerará:

```
In[47]:=x=1.+$MachineEpsilon
Out[47]=1.
In[48]:=x-1.
Out[48]=2.22045×10-16
```

Ejercicio 1. Usa la aritmética de cuatro cifras con redondeo para obtener aproximaciones de las soluciones de las siguientes ecuaciones de segundo grado usando la fórmula de resolución. Calcula además los errores absoluto y relativo de cada una. *Pista: para mejorar la precisión de una de las soluciones de la ecuación de segundo grado, puede ser necesario racionalizar el numerador.*

Ejercicio 2. Repite el ejercicio anterior considerando la aritmética de cuatro cifras con truncamiento.

1. Evalúa $f(2.279)$, calculando primero las potencias de 2.279 y empleando la aritmética de cuatro cifras con redondeo.
2. Evalúa $f(2.279)$, usando la aritmética de cuatro cifras con redondeo y la siguiente expresión de f :

3. Determina el número de operaciones que se realizan en cada uno de los apartados anteriores.
4. Calcula los errores absoluto y relativo de cada aproximación, indicando razonadamente cuál de ellas tiene mayor exactitud.

Ejercicio 5. Obtén la representación computacional (base 2) de los siguientes números enteros:

Ejercicio 6. Calcula a qué números enteros correspondientes a las siguientes palabras de 32 bits:

Ejercicio 7. Indica a qué número real corresponde el número de precisión doble (64 bits) que se indica a continuación:

- 7

Ejercicio 8. Determina la expresión en punto flotante de los siguientes números decimales:

- (a) 7.1 (b) -45.8 (c) 125.74

Ejercicio 9. Determina cuál es el número de condición de las siguientes expresiones, indicando para qué valores la expresión está bien condicionada y para cuáles está mal condicionada:

(a) $f(x) = (x - 1) \cdot \sqrt{x}, \forall x \in [0, +\infty)$ (b) $f(x) = (x - 1)^2, \forall x \in \mathbb{R}.$

(c) $f(x) = x^{-1} \cdot e^x, \forall x \in \mathbb{R} \setminus \{0\}.$

Problemas de ampliación

Ejercicio 10. Usa la aritmética de cuatro cifras con redondeo para obtener aproximaciones de las soluciones de las siguientes ecuaciones de segundo grado usando la fórmula de resolución. Calcula además los errores absoluto y relativo de cada una. *Pista: para mejorar la precisión de una de las soluciones puede ser necesario racionalizar el numerador.*

(a) $\frac{1}{3}x^2 + \frac{123}{4}x - \frac{1}{6} = 0.$ (b) $1.002x^2 + 11.01x + 0.01265 = 0.$

Ejercicio 11. Repite el ejercicio anterior considerando la aritmética de cuatro cifras con truncamiento.

Ejercicio 12. Utilizando el método de Gauss y empleando la aritmética de cuatro cifras con redondeo, resuelve los siguientes sistemas. Estudia además los errores absoluto y relativo del resultado:

(a) $\begin{cases} 1.014x & -6.198y & = 14.23 \\ -18.05x & +112.5y & = -0.1376 \end{cases}$ (b) $\begin{cases} 1.114x & -6.019y & = 13.53 \\ -8.114x & +11.25y & = -1.376 \end{cases}$

Ejercicio 13. Repite el ejercicio anterior considerando ahora el truncamiento en lugar del redondeo.

Ejercicio 14. Considera la función $f(x) = 1.01e^{4x} - 4.62e^{3x} - 3.11e^{2x} + 12.2e^x - 1.99.$

1. Calcula el número de operaciones que hay que realizar para obtener un valor aproximado de $f(2.191).$
2. Halla una aproximación de $f(2.191)$ utilizando la aritmética de cuatro cifras con redondeo.
3. Determina los errores absoluto y relativo de la aproximación obtenida en el apartado anterior.
4. ¿Cuántos cálculos se ahorrarían si se escribiese la función de forma anidada?
5. Calcula la aproximación de $f(2.191)$ con la función expresada de ese modo.
6. Determina los errores absoluto y relativo de la aproximación obtenida en el apartado anterior.

Ejercicio 15. Determina cuál es el número de multiplicaciones y sumas que se necesitan para calcular un sumatorio de la forma:

$$\sum_{i=1}^n \sum_{j=1}^i a_i b_j.$$

Modifica este sumatorio para obtener una expresión equivalente en la que se reduzca el número de operaciones. *Pista: intenta usar una expresión análoga a la usada para escribir los polinomios de manera anidada.*

Ejercicio 16. Diseña e implementa un algoritmo con Mathematica cuyas entradas sean un número entero $n \geq 1$ y los números $x_0, x_1, \dots, x_n$ y x , siendo la salida el resultado del producto $(x - x_0)(x - x_1) \cdots (x - x_n)$.

Ejercicio 17. Para calcular el valor de $(7 - 4\sqrt{3})^4$, utilizaremos el valor aproximado $\sqrt{3} \approx 1.73205$ (con todas sus cifras decimales exactas) y las fórmulas que se indicarán a continuación. Justifica razonadamente cuál de ellas es la más conveniente desde un punto de vista numérico:

$$(7 - 4\sqrt{3})^4 = \frac{1}{(7 + 4\sqrt{3})^4} = \frac{1}{(97 + 56\sqrt{3})^2} = (97 - 56\sqrt{3})^2 = 18817 - 10864\sqrt{3} = \frac{1}{18817 + 10864\sqrt{3}}$$

Ejercicio 18. Implementa con Mathematica una rutina que permita calcular cualquier término de la sucesión de Fibonacci:

$$F(n) = \frac{1}{\sqrt{5}} \left[\left(\frac{1 + \sqrt{5}}{2} \right)^n - \left(\frac{1 - \sqrt{5}}{2} \right)^n \right].$$

Calcula el valor de la aproximación de los primeros seis términos de la sucesión con el comando `N` de Mathematica con la precisión que trae por defecto. Indica razonadamente cuál es la exactitud y la precisión de cada aproximación.

Ejercicio 19. Obtén expresiones estables (i.e. que no produzcan cancelaciones indeseadas) para calcular las siguientes cantidades a partir de una aproximación de x :

1. $\frac{1}{1 + 2x} - \frac{1 - x}{1 + x}$ para $|x| \ll 1$.
2. $\frac{1 - \cos(x)}{x}$ para $|x| \ll 1$.

Ejercicio 20. Obtén la representación computacional en base 2 de los siguientes números enteros:

$$(a) -105 \quad (b) 45 \quad (c) -1005$$

Ejercicio 21. Calcula a qué números enteros correspondientes a las siguientes palabras de 32 bits:

$$(a) 01101000010001000000000000000000 \quad (b) 11101000011100000000000000000000 \\ (c) 10111100000010000000000000000000$$

Ejercicio 22. Indica a qué número real corresponde el número de precisión doble (64 bits) que se indica a continuación:

1. 1 00110010001 101100011000
2. 0 01100100000 000110110100

Ejercicio 23. Determina la expresión en punto flotante de los siguientes números decimales:

- (a) 5.7 (b) 112.56 (c) -25.54

Ejercicio 24. Determina cuál es el número de condición de las siguientes expresiones, indicando para qué valores la expresión está bien condicionada y para cuáles está mal condicionada:

- (a) $f(x) = \log(x - 1)$, $\forall x \in (0, +\infty)$ (b) $f(x) = e^x \cdot x^2$, $\forall x \in \mathbb{R}$.
(c) $f(x) = x^{-1} \cdot \sqrt{x - 1}$, $\forall x \in [1, +\infty)$.